WEEK-3

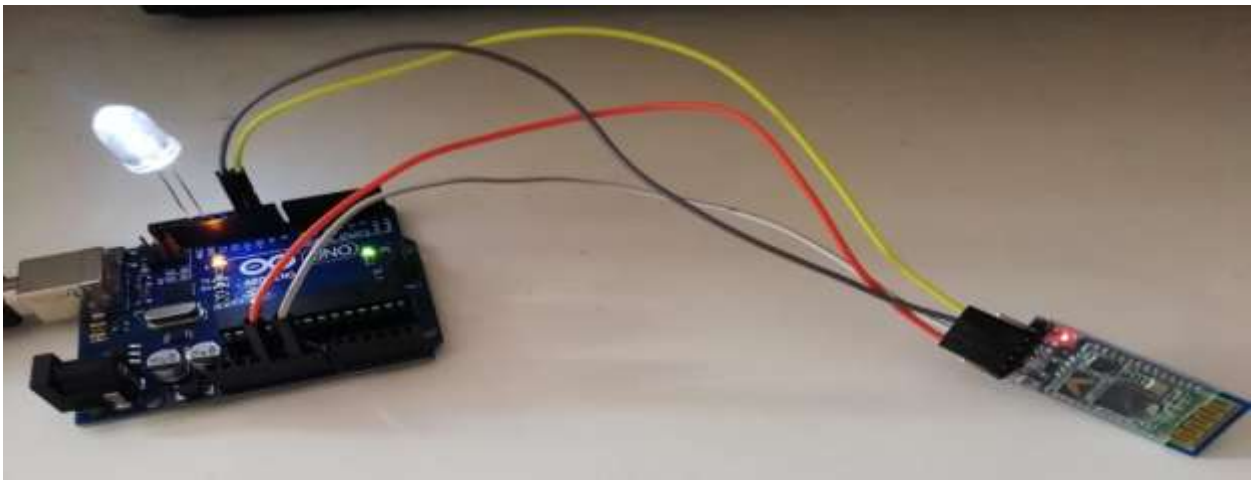
AIM: To interface Bluetooth with Arduino/Raspberry Pi and write a program to turn LED ON/OFF when '1'/'0' is received from smart phone using Bluetooth.

SOURCE CODE:

```
#include <SoftwareSerial.h>
SoftwareSerial Bluetooth(10, 9); // RX, TX
int LED = 13; // the on-board LED
int Data; // the data received



void setup() {
  Bluetooth.begin(9600);
  Serial.begin(9600);
  Serial.println("Waiting for command...");
  Bluetooth.println("Send 1 to turn on the LED. Send 0 to turn Off");
  pinMode(LED,OUTPUT);
}
void loop() {
  if (Bluetooth.available()){ //wait for data received
  Data=Bluetooth.read();
   if(Data=='1'){
   digitalWrite(LED,1);
   Serial.println("LED On!");
   Bluetooth.println("LED On!");
    }
    else if(Data=='0'){
    digitalWrite(LED,0);
    Serial.println("LED Off!");
     Bluetooth.println("LED  On D13 Off ! ");
    }
    else{;}
```

```
  }
delay(100);
}
```

OUTPUT:

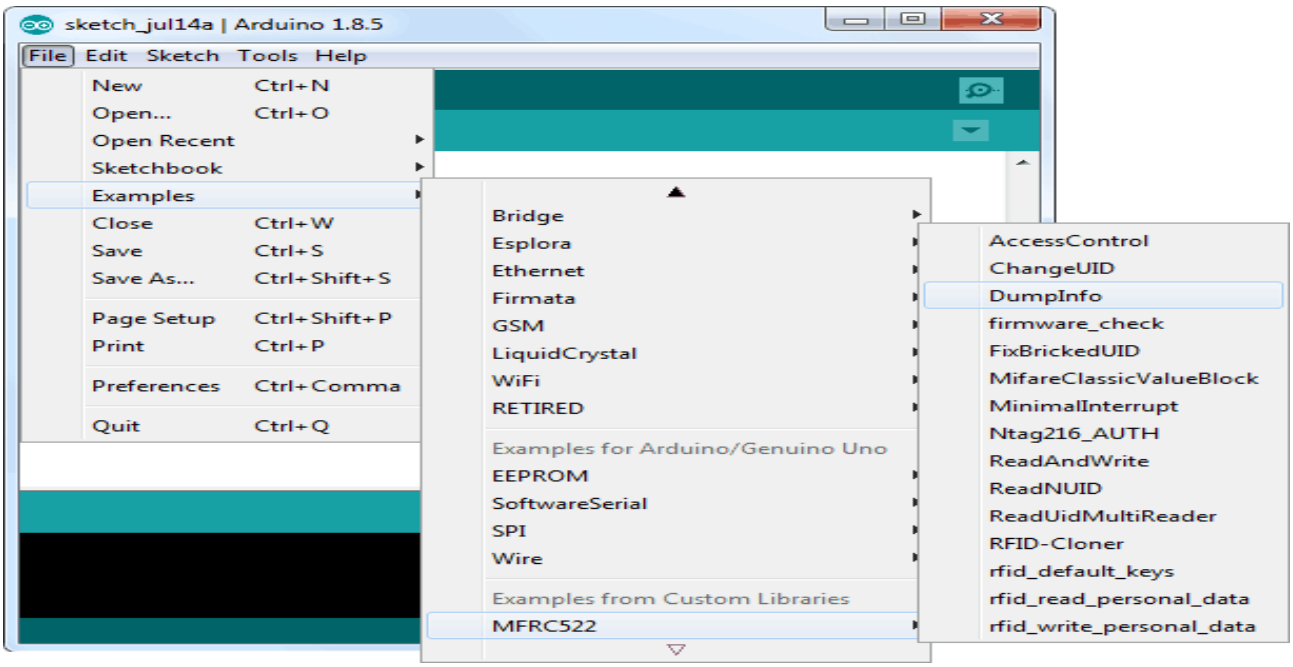


WEEK-4

AIM: Write a program to implement RFID using Arduino.

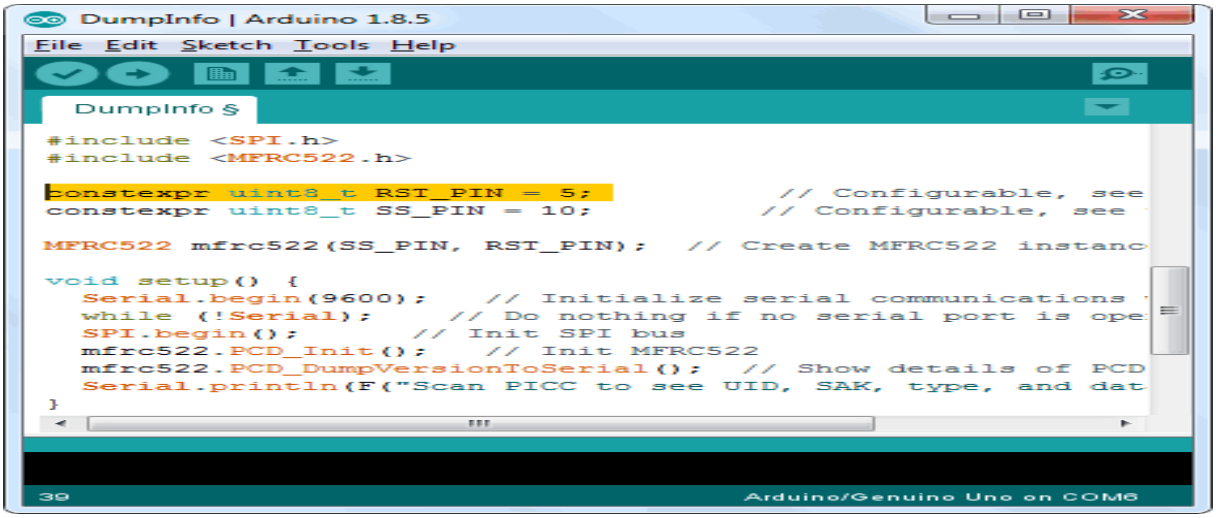
Arduino Code – Reading an RFID Tag

Once you have installed the library, open the Examples submenu and choose MFRC522 > DumpInfo example sketch.



This sketch just reads the tag and displays the information stored in it. This sketch can be very handy before trying out any new tags!

Go to the beginning of the sketch and make sure RST_PIN is initialized correctly, in our case we are using digital pin #5 so change it to 5.



Now upload the sketch and open Serial Monitor. As you bring the tag closer to the module, you'll get something like the following. Do not move the tag until all the information is displayed.

```
#include <SPI.h>
#include <MFRC522.h>

#define RST_PIN         9          // Configurable, see typical pin layout above
#define SS_PIN          10         // Configurable, see typical pin layout above

MFRC522 mfrc522(SS_PIN, RST_PIN);  // Create MFRC522 instance

void setup() {
        Serial.begin(9600);             // Initialize serial communications with the PC
        while (!Serial);                // Do nothing if no serial port is opened (added for Arduinos based on ATMEGA32U4)
        SPI.begin();                    // Init SPI bus
        mfrc522.PCD_Init();             // Init MFRC522
        delay(4);                               // Optional delay. Some board do need more time after init to be ready, see Readme
        mfrc522.PCD_DumpVersionToSerial();      // Show details of PCD - MFRC522 Card Reader details
        Serial.println(F("Scan PICC to see UID, SAK, type, and data blocks..."));
}

void loop() {
        // Reset the loop if no new card present on the sensor/reader. This saves the entire process when idle.
        if ( ! mfrc522.PICC_IsNewCardPresent()) {
                return;
        }
```

```
	// Select one of the cards
	if ( ! mfrc522.PICC_ReadCardSerial()) {
		return;
	}

	// Dump debug info about the card; PICC_HaltA() is automatically called
	mfrc522.PICC_DumpToSerial(&(mfrc522.uid));
}
```

Reading Data from Card:

```
#include <SPI.h>
#include <MFRC522.h>

#define RST_PIN		9		// Configurable, see typical pin layout above
#define SS_PIN		10		// Configurable, see typical pin layout above

MFRC522 mfrc522(SS_PIN, RST_PIN);	// Create MFRC522 instance

//*****************************************************************************
*************//
void setup() {
  Serial.begin(9600);                           // Initialize serial communications with the PC
  SPI.begin();                                  // Init SPI bus
  mfrc522.PCD_Init();                                   // Init MFRC522 card
  Serial.println(F("Read personal data on a MIFARE PICC:"));    //shows in serial that it is ready
to read
}

//*****************************************************************************
*************//
void loop() {

  // Prepare key - all keys are set to FFFFFFFFFFFFh at chip delivery from the factory.
  MFRC522::MIFARE_Key key;
  for (byte i = 0; i < 6; i++) key.keyByte[i] = 0xFF;

  //some variables we need
  byte block;
  byte len;
  MFRC522::StatusCode status;

  //-----------------------------------------

  // Reset the loop if no new card present on the sensor/reader. This saves the entire process when
idle.
  if ( ! mfrc522.PICC_IsNewCardPresent()) {
    return;
```

```
  }

  // Select one of the cards
  if ( ! mfrc522.PICC_ReadCardSerial()) {
    return;
  }

  Serial.println(F("**Card Detected:**"));

  //-------------------------------------------

  mfrc522.PICC_DumpDetailsToSerial(&(mfrc522.uid)); //dump some details about the card

  //mfrc522.PICC_DumpToSerial(&(mfrc522.uid));      //uncomment this to see all blocks in hex

  //-------------------------------------------

  Serial.print(F("Name: "));

  byte buffer1[18];

  block = 4;
  len = 18;

  //------------------------------------------- GET FIRST NAME
  status = mfrc522.PCD_Authenticate(MFRC522::PICC_CMD_MF_AUTH_KEY_A, 4, &key,
&(mfrc522.uid)); //line 834 of MFRC522.cpp file
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("Authentication failed: "));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }

  status = mfrc522.MIFARE_Read(block, buffer1, &len);
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("Reading failed: "));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }

  //PRINT FIRST NAME
  for (uint8_t i = 0; i < 16; i++)
  {
    if (buffer1[i] != 32)
    {
      Serial.write(buffer1[i]);
```

```
    }
  }
  Serial.print(" ");

  //---------------------------------------- GET LAST NAME

  byte buffer2[18];
  block = 1;

  status = mfrc522.PCD_Authenticate(MFRC522::PICC_CMD_MF_AUTH_KEY_A, 1, &key,
&(mfrc522.uid)); //line 834
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("Authentication failed: "));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }

  status = mfrc522.MIFARE_Read(block, buffer2, &len);
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("Reading failed: "));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }

  //PRINT LAST NAME
  for (uint8_t i = 0; i < 16; i++) {
    Serial.write(buffer2[i] );
  }


  //----------------------------------------

  Serial.println(F("\n**End Reading**\n"));

  delay(1000); //change value if you want to read cards faster

  mfrc522.PICC_HaltA();
  mfrc522.PCD_StopCrypto1();
}
```

Writing Data from Card:

```
#include <SPI.h>
#include <MFRC522.h>

#define RST_PIN         9           // Configurable, see typical pin layout above
#define SS_PIN          10          // Configurable, see typical pin layout above
```

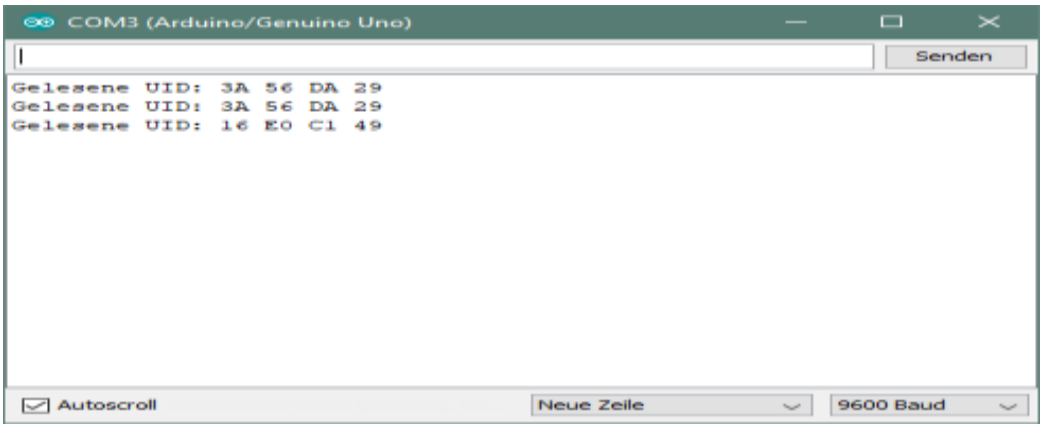
```
MFRC522 mfrc522(SS_PIN, RST_PIN);   // Create MFRC522 instance

void setup() {
  Serial.begin(9600);        // Initialize serial communications with the PC
  SPI.begin();               // Init SPI bus
  mfrc522.PCD_Init();        // Init MFRC522 card
  Serial.println(F("Write personal data on a MIFARE PICC "));
}

void loop() {

  // Prepare key - all keys are set to FFFFFFFFFFFFh at chip delivery from the factory.
  MFRC522::MIFARE_Key key;
  for (byte i = 0; i < 6; i++) key.keyByte[i] = 0xFF;

  // Reset the loop if no new card present on the sensor/reader. This saves the entire process when
idle.
  if ( ! mfrc522.PICC_IsNewCardPresent()) {
    return;
  }

  // Select one of the cards
  if ( ! mfrc522.PICC_ReadCardSerial()) {
    return;
  }

  Serial.print(F("Card UID:"));    //Dump UID
  for (byte i = 0; i < mfrc522.uid.size; i++) {
    Serial.print(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " ");
    Serial.print(mfrc522.uid.uidByte[i], HEX);
  }
  Serial.print(F(" PICC type: "));   // Dump PICC type
  MFRC522::PICC_Type piccType = mfrc522.PICC_GetType(mfrc522.uid.sak);
  Serial.println(mfrc522.PICC_GetTypeName(piccType));

  byte buffer[34];
  byte block;
  MFRC522::StatusCode status;
  byte len;
  Serial.setTimeout(20000L) ;     // wait until 20 seconds for input from serial
  // Ask personal data: Family name
  Serial.println(F("Type Family name, ending with #"));
  len = Serial.readBytesUntil('#', (char *) buffer, 30) ; // read family name from serial
  for (byte i = len; i < 30; i++) buffer[i] = ' ';     // pad with spaces

  block = 1;
```

```
  //Serial.println(F("Authenticating using key A...")); 
  status = mfrc522.PCD_Authenticate(MFRC522::PICC_CMD_MF_AUTH_KEY_A, block, 
&key, &(mfrc522.uid)); 
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("PCD_Authenticate() failed: ")); 
    Serial.println(mfrc522.GetStatusCodeName(status)); 
    return;
  }
  else Serial.println(F("PCD_Authenticate() success: "));

  // Write block
  status = mfrc522.MIFARE_Write(block, buffer, 16); 
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("MIFARE_Write() failed: ")); 
    Serial.println(mfrc522.GetStatusCodeName(status)); 
    return;
  }
  else Serial.println(F("MIFARE_Write() success: "));

  block = 2;
  //Serial.println(F("Authenticating using key A...")); 
  status = mfrc522.PCD_Authenticate(MFRC522::PICC_CMD_MF_AUTH_KEY_A, block, 
&key, &(mfrc522.uid)); 
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("PCD_Authenticate() failed: ")); 
    Serial.println(mfrc522.GetStatusCodeName(status)); 
    return;
  }
  status = mfrc522.MIFARE_Write(block, &buffer[16], 16); 
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("MIFARE_Write() failed: ")); 
    Serial.println(mfrc522.GetStatusCodeName(status)); 
    return;
  }
  else Serial.println(F("MIFARE_Write() success: "));
  // Ask personal data: First name
  Serial.println(F("Type First name, ending with #")); 
  len = Serial.readBytesUntil('#', (char *) buffer, 20) ; // read first name from serial
  for (byte i = len; i < 20; i++) buffer[i] = ' ';     // pad with spaces

  block = 4;
  //Serial.println(F("Authenticating using key A...")); 
  status = mfrc522.PCD_Authenticate(MFRC522::PICC_CMD_MF_AUTH_KEY_A, block, 
&key, &(mfrc522.uid)); 
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("PCD_Authenticate() failed: "));
```

```
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }

  // Write block
  status = mfrc522.MIFARE_Write(block, buffer, 16);
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("MIFARE_Write() failed: "));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }
  else Serial.println(F("MIFARE_Write() success: "));
  block = 5;
  //Serial.println(F("Authenticating using key A..."));
  status = mfrc522.PCD_Authenticate(MFRC522::PICC_CMD_MF_AUTH_KEY_A, block,
&key, &(mfrc522.uid));
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("PCD_Authenticate() failed: "));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }
  status = mfrc522.MIFARE_Write(block, &buffer[16], 16);
  if (status != MFRC522::STATUS_OK) {
    Serial.print(F("MIFARE_Write() failed: "));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }
  else Serial.println(F("MIFARE_Write() success: "));
  Serial.println(" ");
  mfrc522.PICC_HaltA(); // Halt PICC
  mfrc522.PCD_StopCrypto1();  // Stop encryption on PCD}
```

Arduino Code – Reading an RFID Tag

OUTPUT:



WEEK-5

AIM: Write a program to monitor temperature and humidity using Arduino and Raspberry Pi.

Arduino Source Code:

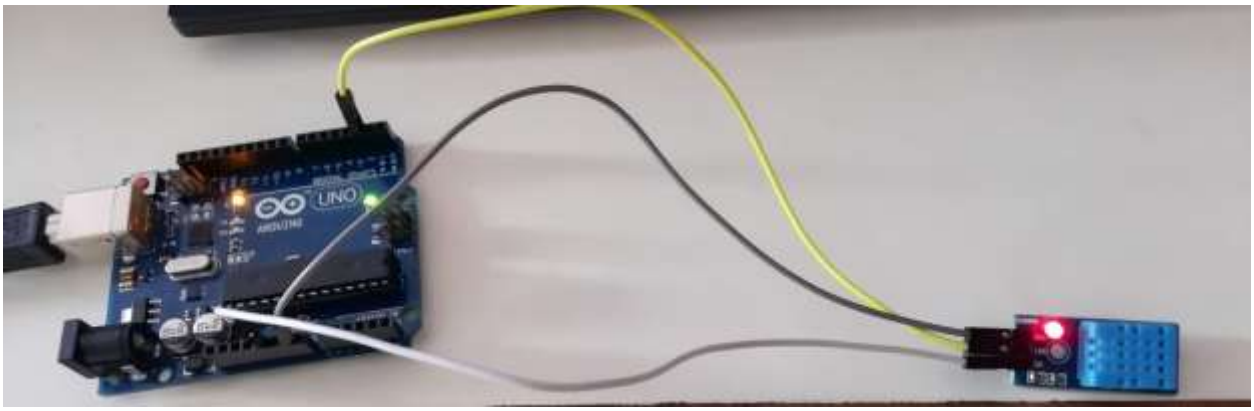
```
#include "DHT.h"
#define DHTPIN 2
//#define DHTTYPE DHT11   // DHT 11
#define DHTTYPE DHT11   // DHT 22  (AM2302), AM2321
//#define DHTTYPE DHT21   // DHT 21 (AM2301)
DHT dht(DHTPIN, DHTTYPE);
void setup() {
 Serial.begin(9600);
Serial.println(F("DHTxx test!"));
dht.begin();
}
void loop() {
 delay(2000);
  float h = dht.readHumidity();
  // Read temperature as Celsius (the default)
  float t = dht.readTemperature();
// Read temperature as Fahrenheit (isFahrenheit = true)
  float f = dht.readTemperature(true);
  if (isnan(h) || isnan(t) || isnan(f)) {
Serial.println(F("Failed to read from DHT sensor!"));
    return;
  }
  float hif = dht.computeHeatIndex(f, h);
  float hic = dht.computeHeatIndex(t, h, false);
  Serial.print(F("Humidity: "));
  Serial.print(h);
  Serial.print(F("%  Temperature: "));
```

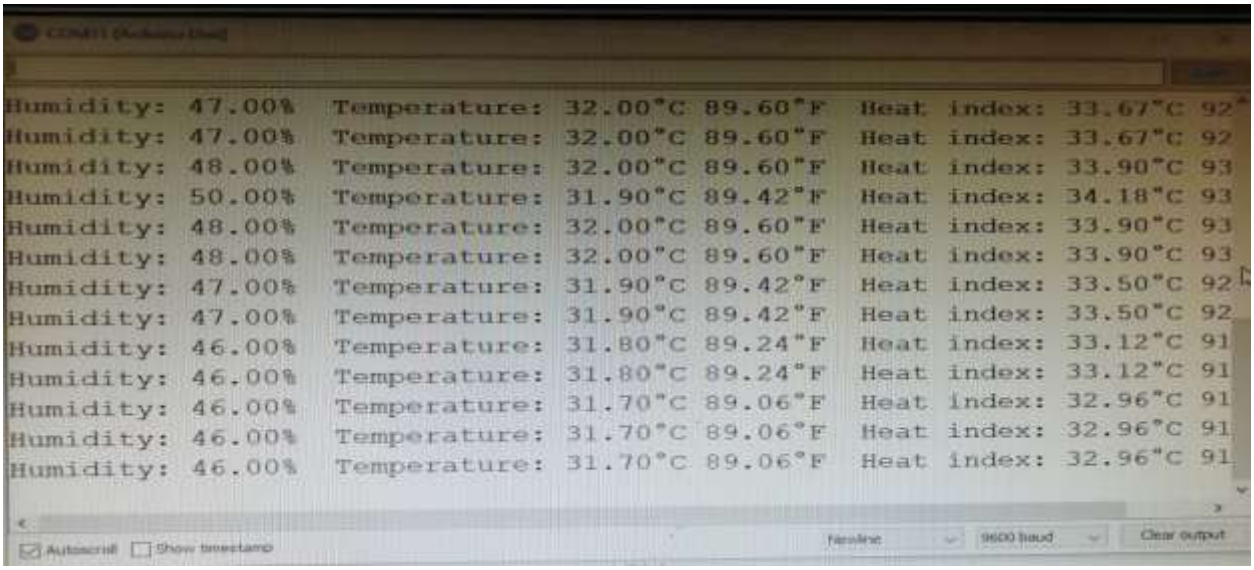
```
  Serial.print(t);
  Serial.print(F("°C "));
  Serial.print(f);
  Serial.print(F("°F  Heat index: "));
  Serial.print(hic);
  Serial.print(F("°C "));
  Serial.print(hif);
  Serial.println(F("°F"));
}
```

RaspberryPi Source Code

```
import time
import board
import adafruit_dht
dhtDevice = adafruit_dht.DHT11(board.D22,use_pulseio=False)

try:
    while True:
        # Print the values to the serial port
        temperature_c = dhtDevice.temperature
        temperature_f = temperature_c * (9 / 5) + 32
        humidity = dhtDevice.humidity
        print(
            "Temp:  {:.1f} C    Humidity: {}% ".format(
                temperature_c, humidity
            )
        )

except RuntimeError as error:
        # Errors happen fairly often, DHT's are hard to read, just keep
going
        print(error.args[0])
        time.sleep(2.0)
```

OUTPUT:


UNO


Humidity: 47.00% Temperature: 32.00°C 89.60°F Heat index: 33.67°C 92
Humidity: 47.00% Temperature: 32.00°C 89.60°F Heat index: 33.67°C 92
Humidity: 48.00% Temperature: 32.00°C 89.60°F Heat index: 33.90°C 93
Humidity: 50.00% Temperature: 31.90°C 89.42°F Heat index: 34.18°C 93
Humidity: 48.00% Temperature: 32.00°C 89.60°F Heat index: 33.90°C 93
Humidity: 48.00% Temperature: 32.00°C 89.60°F Heat index: 33.90°C 93
Humidity: 47.00% Temperature: 31.90°C 89.42°F Heat index: 33.50°C 92
Humidity: 47.00% Temperature: 31.90°C 89.42°F Heat index: 33.50°C 92
Humidity: 46.00% Temperature: 31.80°C 89.24°F Heat index: 33.12°C 91
Humidity: 46.00% Temperature: 31.80°C 89.24°F Heat index: 33.12°C 91
Humidity: 46.00% Temperature: 31.70°C 89.06°F Heat index: 32.96°C 91
Humidity: 46.00% Temperature: 31.70°C 89.06°F Heat index: 32.96°C 91
Humidity: 46.00% Temperature: 31.70°C 89.06°F Heat index: 32.96°C 91
Autoscroll
Show timestamp
Newline
9600 baud
Clear output

WEEK-6

Aim: Write a program to monitor Sensor data using Arduino using IoT Cloud application.

SOURCE CODE:

```
#include <ESP8266WiFi.h>
String apiKey = "xxx";
const char *ssid =  "xxx";
const char *pass =  "xxx";
const char* server = "api.thingspeak.com";
#define IRpin D2
WiFiClient client;
int value;
void setup()
{
Serial.begin(115200);
pinMode(IRpin, INPUT);
delay(1000);
Serial.println("Connecting to ");
Serial.println(ssid);
WiFi.begin(ssid, pass);
while (WiFi.status() != WL_CONNECTED)
{
delay(2000);
Serial.print(".");
}
Serial.println("   ");
Serial.println("WiFi connected");
}
void loop(){
 value = digitalRead(IRpin);
Serial.println(value);
  if(value==0)
  {
  Serial.print("object detected");
  }
  else
  {
  Serial.print("no object detected");
  }
if (client.connect(server,80))
{
String postStr = apiKey;
postStr +="&field1=";
postStr += String(value);
postStr += "\r\n\r\n";

client.print("POST /update HTTP/1.1\n");
client.print("Host: api.thingspeak.com\n");
client.print("Connection: close\n");
client.print("X-THINGSPEAKAPIKEY: "+apiKey+"\n");
```

```
client.print("Content-Type: application/x-www-form-urlencoded\n");
client.print("Content-Length: ");
client.print(postStr.length());
client.print("\n\n");
client.print(postStr);
client.stop();
Serial.println("Waiting...");
delay(1000); }}
```

OUTPUT:

Uploading the data in CLOUD:

Visualizing the output in Serial monitor:

WEEK-7

Aim: Upload temperature and humidity data to the cloud using Arduino or Raspberry Pi.

SOURCE CODE:

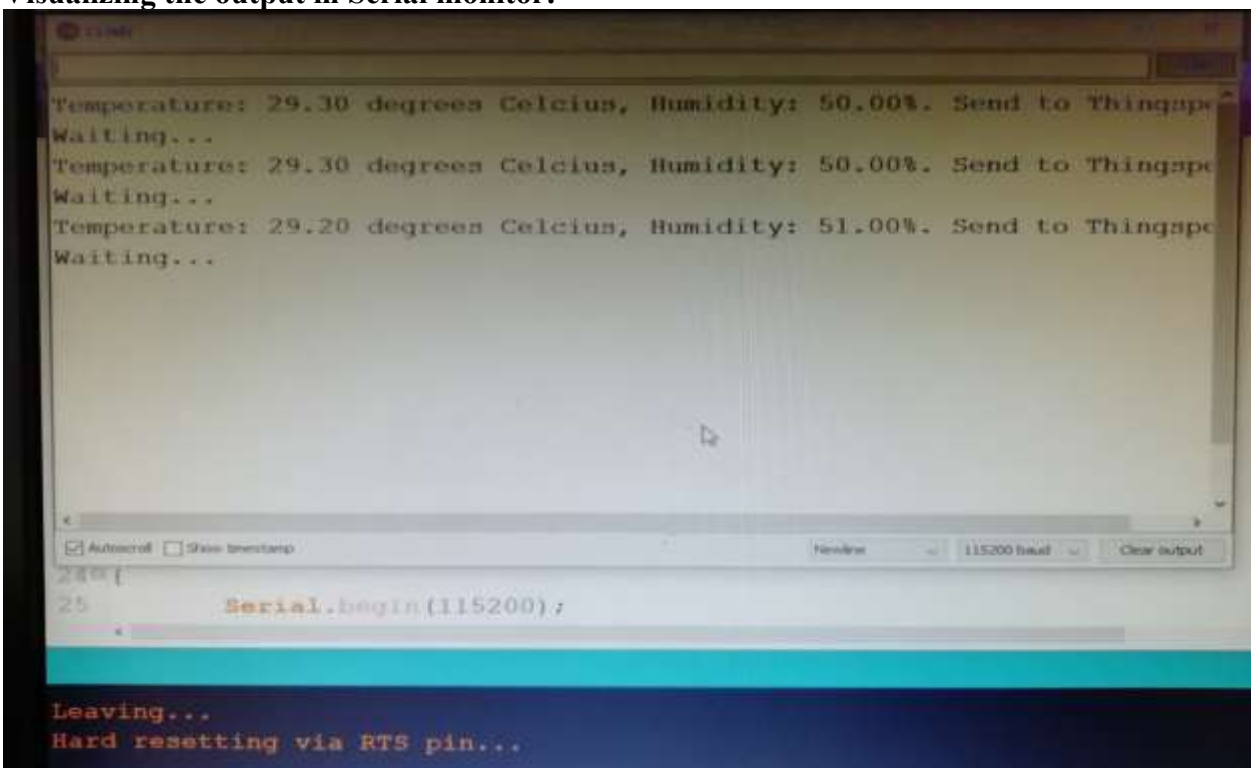
```
#include <DHT.h>
#include <DHT_U.h>
#include <ESP8266WiFi.h>
String apiKey = "ZYUP9R7N15OEBYRO";  //
const char *ssid =  "xxx";
const char *pass =  "xxx";
const char* server = "api.thingspeak.com";
#define DHTPIN D3
DHT dht(DHTPIN, DHT11);
WiFiClient client;

 void setup()
{
  Serial.begin(115200);
  delay(1000);
  dht.begin();
  Serial.println("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(2000);
    Serial.print(".");
  }
Serial.println("");
Serial.println("WiFi connected");
}
void loop()
{
 float h = dht.readHumidity();
 float t = dht.readTemperature();
 if (isnan(h) || isnan(t))
 {
  Serial.println("Failed to read from DHT sensor!");
  return;
 }
if (client.connect(server,80))   //   "184.106.153.149" or api.thingspeak.com
{
  String postStr = apiKey;
```

```
  postStr +="&field1=";
  postStr += String(t);
  postStr +="&field2=";
  postStr += String(h);
  postStr += "\r\n\r\n";
client.print("POST /update HTTP/1.1\n");
client.print("Host: api.thingspeak.com\n");
client.print("Connection: close\n");
client.print("X-THINGSPEAKAPIKEY: "+apiKey+"\n");
client.print("Content-Type: application/x-www-form-urlencoded\n");
client.print("Content-Length: ");
client.print(postStr.length());
client.print("\n\n");
client.print(postStr);
Serial.print("Temperature: ");
Serial.print(t);
Serial.print(" degrees Celcius, Humidity: ");
Serial.print(h);
Serial.println("%. Send to Thingspeak.");
}
  client.stop();
  Serial.println("Waiting...");
  delay(1000);
}
}
```

OUTPUT:

Uploading the data in CLOUD:

Visualizing the output in Serial monitor:



WEEK-8

Aim: Write a program on Arduino or Raspberry Pi to retrieve temperature and humidity data from the Cloud

Set up in source code:

1. Set your Wi-Fi SSID and password.

2. Set the API Key

3. Save->Compile->upload->now us can visualize our data in cloud

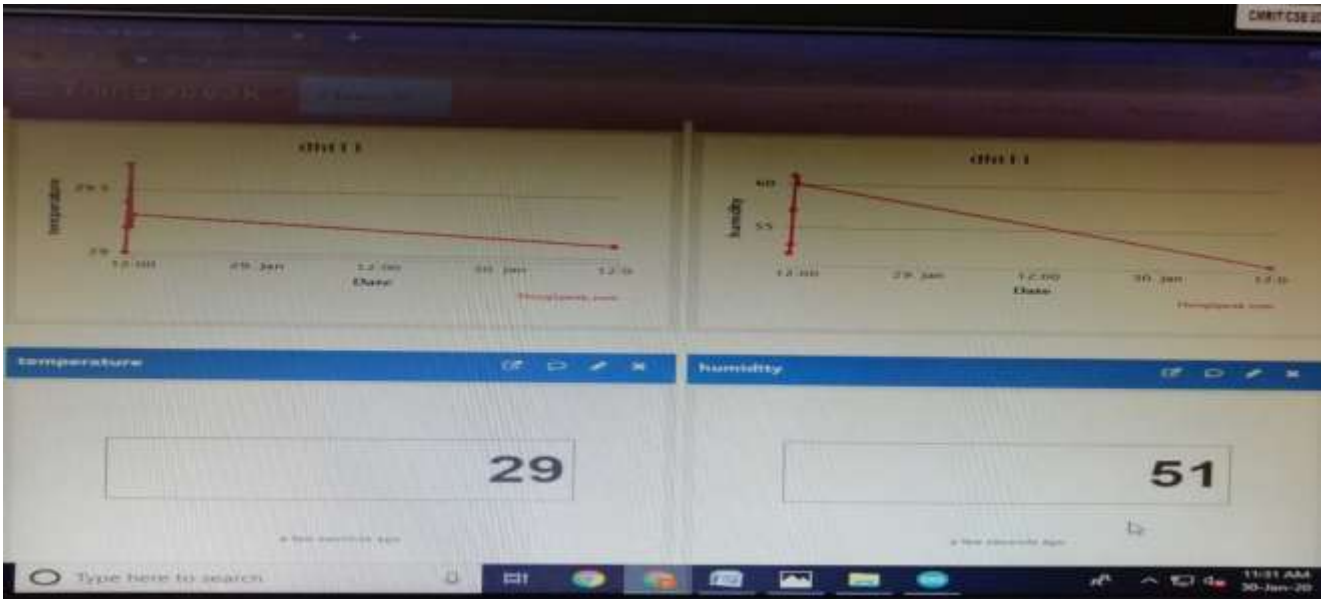
Source Code:

```
#include "ThingSpeak.h"
#include <ESP8266WiFi.h>
#include<DHT.h>
const char ssid[] = "xxx";  // your network SSID (name)
const char pass[] = "xxx";   // your network password
int statusCode = 0;
WiFiClient  client;

//---------Channel Details---------//
unsigned long counterChannelNumber =  2846266;            // Channel ID
const char * myCounterReadAPIKey = "SXJMCVLJWIZMZLLF"; // Read API Key
const int FieldNumber1 = 1;  // The field you wish to read
const int FieldNumber2 = 2;  // The field you wish to read
//-------------------------------//

void setup()
{
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  ThingSpeak.begin(client);
}

void loop()
{
  //----------------- Network -----------------//
  if (WiFi.status() != WL_CONNECTED)
  {
    Serial.print("Connecting to ");
    Serial.print(ssid);
    Serial.println(" ....");
    while (WiFi.status() != WL_CONNECTED)
    {
      WiFi.begin(ssid, pass);
      delay(5000);
```

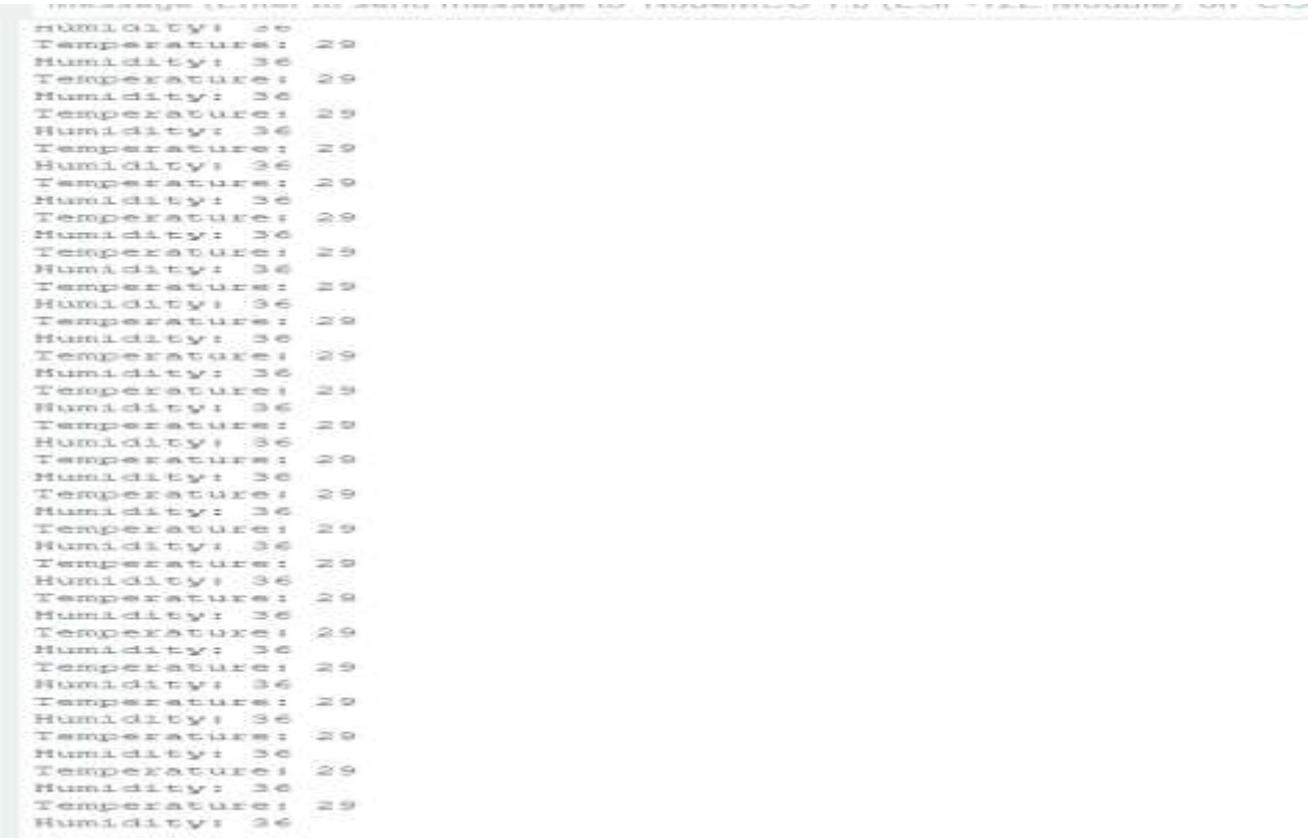
```
    }
    Serial.println("Connected to Wi-Fi Succesfully.");
  }
  //--------- End of Network connection--------//

  //---------------- Channel 1 ----------------//
      long temp = ThingSpeak.readLongField(counterChannelNumber, FieldNumber1,
myCounterReadAPIKey);
  statusCode = ThingSpeak.getLastReadStatus();
  if (statusCode == 200)
  {
    Serial.print("Temperature: ");
    Serial.println(temp);
  }
  else
  {
    Serial.println("Unable to read channel / No internet connection");
  }
  delay(100);
  //-------------- End of Channel 1 -------------//

  //---------------- Channel 2 ----------------//
     long humidity = ThingSpeak.readLongField(counterChannelNumber, FieldNumber2,
myCounterReadAPIKey);
  statusCode = ThingSpeak.getLastReadStatus();
  if (statusCode == 200)
  {
    Serial.print("Humidity: ");
    Serial.println(humidity);
  }
  else
  {
    Serial.println("Unable to read channel / No internet connection");
  }
  delay(100);
  //-------------- End of Channel 2 -------------//
}
```

OUTPUT:

Uploading the data in CLOUD:



Visualizing the output in Serial monitor:



```
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
```

WEEK-9

AIM: Write a program to create TCP Server on cloud using Arduino and Respond with humidity data to TCP Client when requested.

Source code:

```
#include "ESP8266WiFi.h"
#include "DHT.h"
const char* ssid="Galaxy A21sE600";
const char* password ="zilh8480";
WiFiServer wifiServer(8080);
DHT dht(D3, DHT22);
void setup() {
  Serial.begin(115200);
  delay(1000);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting..");
  }
  Serial.print("Connected to WiFi. IP:");
  Serial.println(WiFi.localIP());
  wifiServer.begin();
  dht.begin();
}
void loop() {
   WiFiClient client = wifiServer.available();
   if (client) {
     while (client.connected()) {
       while (client.available()>0) {
        float t=dht.readTemperature();
        float h = dht.readHumidity();
        client.print("humidity :");
```

```
      client.print("temperature :");
      client.println(h);
      Serial.println(h);
      client.println(t);
      Serial.println(t);
      delay(2000);
    }
  }
  client.stop();
  Serial.println("Client disconnected");
 }
}
```

OUTPUT:

```
11:40:57.483 -> 33.00
11:40:57.483 -> 31.50
11:40:59.479 -> 33.00
11:40:59.524 -> 31.50
11:41:01.506 -> 33.00
11:41:01.543 -> 31.60
11:41:29.425 -> Connecting..
11:41:30.459 -> Connecting..
11:41:31.446 -> Connecting..
11:41:32.451 -> Connecting..
11:41:32.451 -> Connected to WiFi. IP:192.168.10.102
```

Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
11:39:52.585 -> 32.00
11:39:52.624 -> 31.90
11:39:54.638 -> 32.00
11:39:54.638 -> 31.80
11:39:56.640 -> 32.00
11:39:56.675 -> 31.80
11:39:58.691 -> 32.00
11:39:58.691 -> 31.90
11:40:00.694 -> 32.00
11:40:00.736 -> 31.80
11:40:02.740 -> 32.00
11:40:02.740 -> 31.90
11:40:04.749 -> 32.00
11:40:04.787 -> 31.70
11:40:06.791 -> 32.00
11:40:06.791 -> 31.80
11:40:08.842 -> 32.00
11:40:08.842 -> 31.70
```

11:42
TCP Client
IP address or domain
Port
192.168.10.102
9090
Disconnect
Received:
humidity :
temperature :
32.00
31.90
humidity :
temperature :
32.00
31.70
humidity :
HEX
AutoScroll
Clear
Message for send:
hii
Add CR LF
Send message

WEEK-10

AIM: Write a program to create UDP Server on cloud using Arduino and Respond with humidity data to UDP Client when requested.

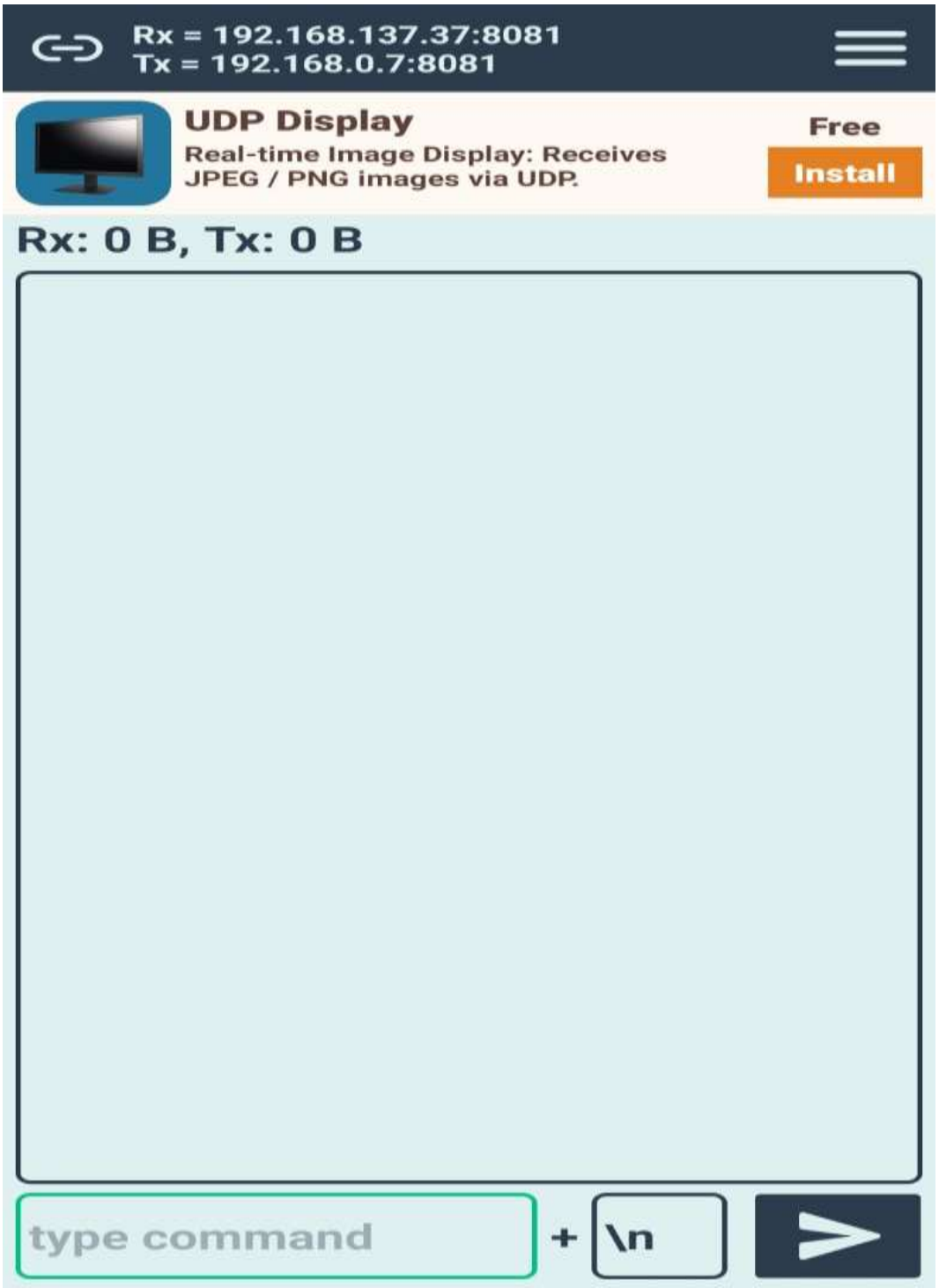
SOURCE CODE:

```
#include <ESP8266WiFi.h>

#include <WiFiUdp.h>

#include <DHT.h>

const char* ssid = "Galaxy A21sE600";

const char* password = "zilh8480";

const char* udpAddress = "192.168.87.225";

const int udpPort = 1234;

#define DHTPIN D3

#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

WiFiUDP udp;

void setup() {

  Serial.begin(115200);

  Serial.println();

  Serial.println("Connecting to WiFi...");

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {

    delay(2000);

    Serial.print(".");

  }

  Serial.println();

  Serial.println("WiFi connected.");

  dht.begin();

}

void loop() {

  delay(2000);

  float temperature = dht.readTemperature();

  float humidity = dht.readHumidity();
```

```
  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.print(" °C\tHumidity: ");
  Serial.print(humidity);
  Serial.println(" %");
  Serial.println("Sending data over UDP...");
  udp.beginPacket(udpAddress, udpPort);
  udp.print("Temperature: ");
  udp.print(temperature);
  udp.print(" °C, Humidity: ");
  udp.print(humidity);
  udp.println(" %");
  udp.endPacket();
  Serial.println("Data sent over UDP.");
}
```

OUTPUT:

```
Output    Serial Monitor ×
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
11:48:10.485 -> Temperature: 29.90 °C   Humidity: 33.00 %
11:48:10.485 -> Sending data over UDP...
11:48:10.485 -> Data sent over UDP.
11:48:20.532 -> Temperature: 29.90 °C   Humidity: 33.00 %
11:48:20.532 -> Sending data over UDP...
11:48:20.532 -> Data sent over UDP.
11:48:30.541 -> Temperature: 29.90 °C   Humidity: 33.00 %
11:48:30.541 -> Sending data over UDP...
11:48:30.541 -> Data sent over UDP.
11:48:40.577 -> Temperature: 29.80 °C   Humidity: 34.00 %
11:48:40.577 -> Sending data over UDP...
11:48:40.577 -> Data sent over UDP.
11:48:50.601 -> Temperature: 29.80 °C   Humidity: 34.00 %
11:48:50.601 -> Sending data over UDP...
11:48:50.601 -> Data sent over UDP.
```


Rx = 192.168.137.37:8081
Tx = 192.168.0.7:8081
UDP Display
Real-time Image Display: Receives JPEG / PNG images via UDP.
Free
Install
Rx: 0 B, Tx: 0 B
type command
+
\n

Rx: 0 B, Tx: 0 B

Temperature : 28.5 °C	Temperature : 28.7°C
Humidity : 65 %	Humidity : 65 %
Temperature : 28.5 °C	Temperature : 28.7°C
Humidity : 65 %	Humidity : 66 %